Assignment 6

David Wilson

CSCI 580 - 01

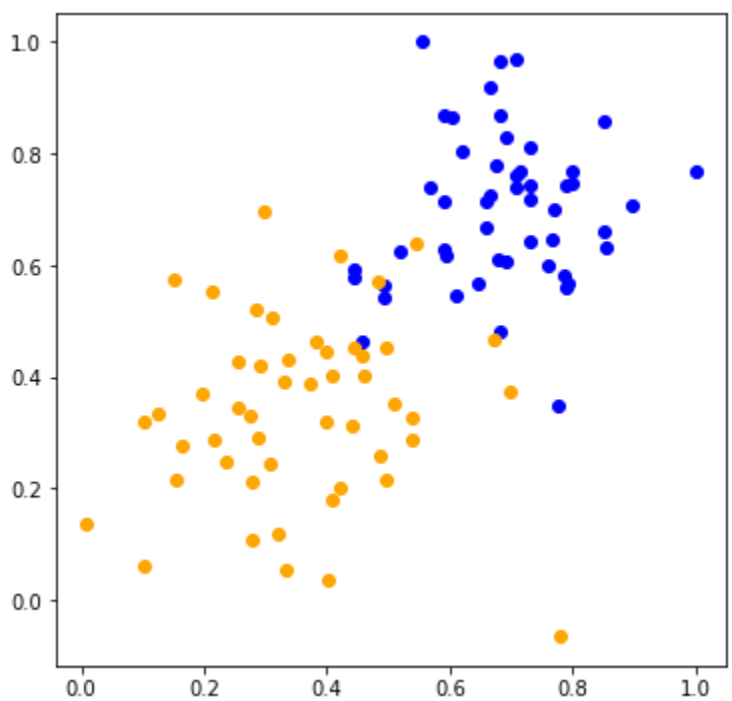
Prof. Bo Shen

Intro

In this assignment, we'll be implementing multiple different approaches in making a Perceptron model. Our first approach is the Heuristic approach, which is the simpler approach, updating weights based on misclassification. However, we will also be implementing the Gradient Descent method, which converges towards a best fit.

Setup

In order to begin this assignment, I created a new Jupyter Notebook file with a basic frame, giving a brief introduction and some code areas. I separated the logic into two separate sections, having one for the Heuristic Perceptron, and another for Gradient Descent. As well as downloading required libraries, I set up the plots to be used as well. Below is a plot with only the given data points, which we'll be using in our code, where blue dots are points where y==0, and orange dots are where y==1.

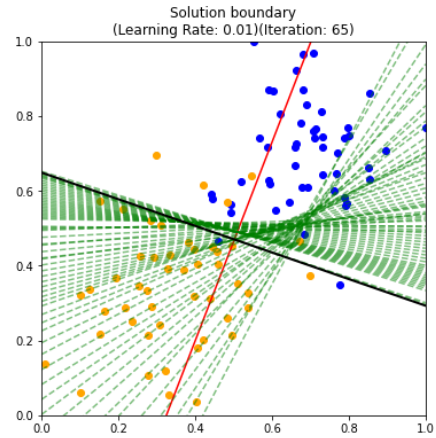


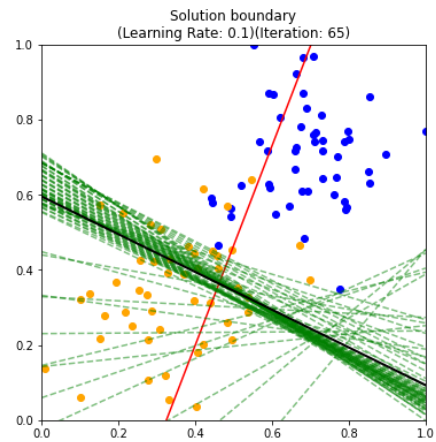
On top of this, I set up learning rate and max epoch variables to be used.

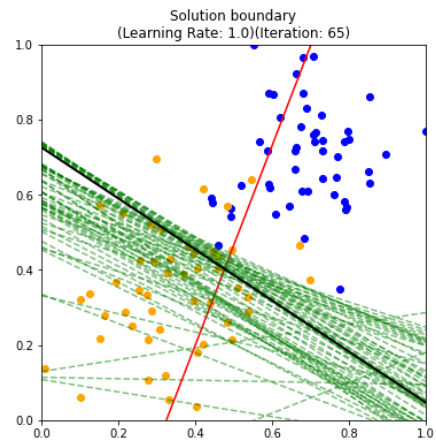
Results & Analysis

Heuristic Approach

Firstly, I implemented a heuristic driven Perceptron, operating within a set ‘epoch count’. Once done, I decided to run it with different learning rates, while maintaining the same random weights/bias in order to better visualize the differences. Overall, each trial seems to reach a viable solution. The difference lies within their step size. Starting with our learning rate of 0.01, we can see that our line does not move much per iteration, smoothly iterating to an ideal solution. On the other end, our trial with a 1.0 learning rate seems to make much more erratic shifts, and while this may reach a solution more quickly (assuming a very off-put start), it could also have a higher chance to overshoot its solution, needing to iterate backwards. As a result, I’m led to believe that the learning rate is not very straightforward, and a higher or lower learning rate is not relative to better success. Rather, it is more dependent on a given situation, and will likely need to be changed based on circumstance.







Heuristic Code

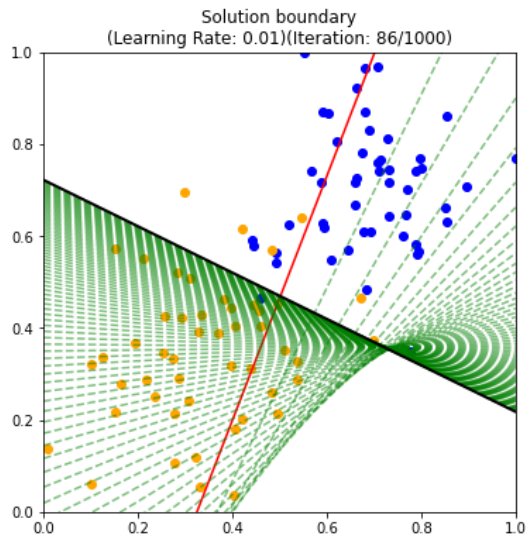
```
np.random.seed(1)

weights = np.random.randn(2)
bias = np.random.randn()

result_history = []

for i in range(epoch_count):
    result_history.append((weights.copy(), bias))
    for x in range(len(p_data)):
        x_0 = p_data[x][0]
        x_1 = p_data[x][1]
        y_true = p_data[x][2]

        y_hat = 1 if weights[0] * x_0 + weights[1] * x_1 + bias >= 0 else 0

        if y_hat != y_true:
            if y_hat == 0:
                bias = bias + learning_rate
                weights[0] = weights[0] + (learning_rate * x_0)
                weights[1] = weights[1] + (learning_rate * x_1)
            else:
                bias = bias - learning_rate
                weights[0] = weights[0] - (learning_rate * x_0)
                weights[1] = weights[1] - (learning_rate * x_1)
    # Print results
    #print(weights, " ", bias)

result_history.append((weights.copy(), bias))
```

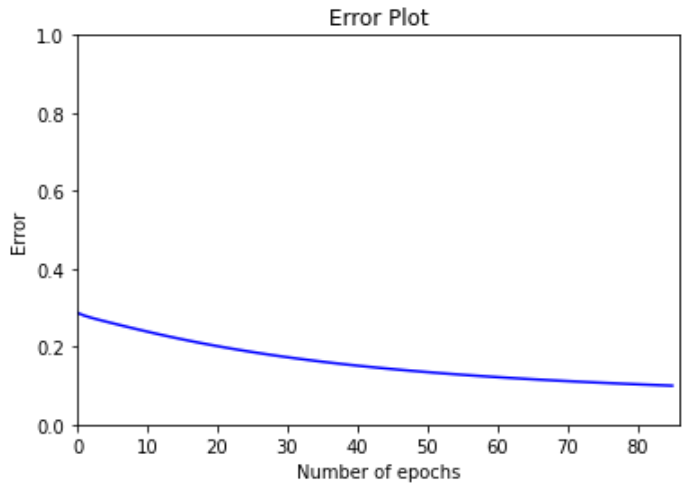
Gradient Descent

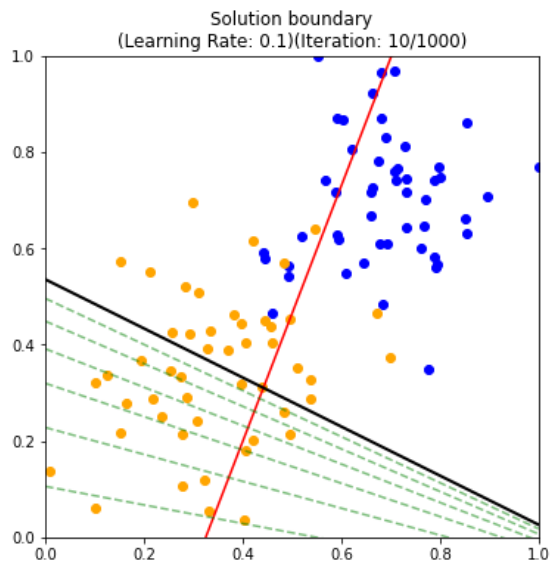
Secondly, I implemented a Gradient Descent approach, which is very similar to the Heuristic approach. However, instead of simply changing bias/weights on misclassifications, we change based on calculated predictions/errors. Using this, we get similar outcomes, but the process shown is quite different. Instead of our erratically paced heuristic approach, the gradient appears more similarly to, well, a gradient, gradually moving towards the most optimal solution. The code has a “max epoch count” similar to our last approach. However, it will stop once the error is small enough.

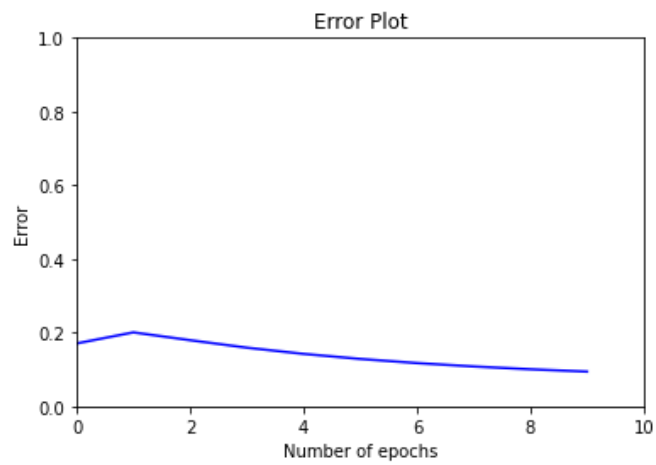
Below we can see the outcomes based on different learning rates, and while we could change the max epoch count, it would likely stop at the same iteration count each time (unless we set max count < optimal iteration index, which then would result in a less optimal solution).

In the images below, we can see how our code runs based on different learning rates. Essentially, we get the same answers that we did with Heuristics. Our higher learning rate means that we take larger leaps towards our desired solution, and in this case, we actually reach it much quicker (or at least, our error quota of <=0.1). From the data, we can see as stated, but also, while our higher learning rate gets us to our solution quicker, it also seems to be much more unstable regarding its error percentage, with it rising in the beginning. Whether this is an issue with the function or with my code, I’m not sure, but I believe it helps shine a light on the pros and cons of differing learning rates.


Solution boundary
(Learning Rate: 0.01)(Iteration: 86/1000)


Error Plot
Error
Number of epochs


Solution boundary
(Learning Rate: 0.1)(Iteration: 10/1000)


Error Plot
Error
Number of epochs

Gradient Descent Code

```
for i in range(epoch_count):
    result_history.append((weights.copy(), bias))
    total_error = 0.0
    for x in range(len(p_data)):
        x_0 = p_data[x][0]
        x_1 = p_data[x][1]
        y_true = p_data[x][2]

        z = weights[0] * x_0 + weights[1] * x_1 + bias
        y_hat = 1 / (1 + np.exp(-z))

        y_error = y_true - y_hat
        total_error = total_error + y_error**2

        bias = bias + (learning_rate * y_error)
        weights[0] = weights[0] + (learning_rate * y_error * x_0)
        weights[1] = weights[1] + (learning_rate * y_error * x_1)
    total_error = total_error / len(p_data)
    error_history.append(total_error)

    # If error is small enough, break.
    if total_error <= 0.1:
        break
```